# Code The Escape

## Assignment 2 - Binary search and Sorting

### Ques 1 - Count Element Occurence - InterviewBit

Given a sorted array of integers, find the number of occurrences of a given target value.
Your algorithm's runtime complexity must be in the order of O(log n).
If the target is not found in the array, **return 0**
Example :
Given [5, 7, 7, 8, 8, 10] and target value 8,
return 2.

### Ques 2 - Rotated Array - InterviewBit

Suppose a sorted array **A** is rotated at some pivot unknown to you beforehand.
(i.e., **1 2 4 5 6 7** might become **4 5 6 7 1 2**).
Find the minimum element.
The array will not contain duplicates.
**Note**:- Use the circular rotated property of the array to solve the problem.

### Ques 3 - Search in Bitonic Array! - InterviewBit

Given a bitonic sequence **A** of **N** distinct elements, write a program to find a given element **B** in the bitonic sequence in **O(logN)** time.
**NOTE:**
A Bitonic Sequence is a sequence of numbers which is first strictly increasing then after a point strictly decreasing.

### Ques 4 - WoodCutting Made Easy! - InterviewBit

There is given an integer array **A** of size **N** denoting the heights of **N** trees.
Lumberjack Ojas needs to chop down **B** metres of wood. It is an easy job for him since he has a nifty new woodcutting machine that can take down forests like wildfire. However, Ojas is only allowed to cut a single row of trees.
Ojas's machine works as follows: Ojas sets a height parameter **H** (in metres), and the machine raises a giant sawblade to that height and cuts off all tree parts higher than **H** (of course, trees not higher than **H** meters remain intact). Ojas then takes the parts that were cut off. For example, if the tree row contains trees with heights of 20, 15, 10, and 17 metres, and Ojas raises his sawblade to 15 metres, the remaining tree heights after cutting will be 15, 15, 10, and 15 metres, respectively, while Ojas will take 5 metres off the first tree and 2 metres off the fourth tree (7 metres of wood in total).
Ojas is **ecologically** minded, so he doesn't want to cut off more wood than necessary. That's why he wants to set his sawblade as high as possible. Help Ojas find the **maximum integer height** of the sawblade that still allows him to cut off **at least B** metres of wood.
**NOTE:**
The sum of all heights will exceed **B**, thus Ojas will always be able to obtain the required amount of wood

**Ques 5 -** **Sorted Insert Position - InterviewBit**

Given a sorted array **A** and a target value **B**, return the index if the target is found. If not, return the index where it would be if it were inserted in order.
You may assume no duplicates in the array.

**Ques 6 -** **Matrix Search - InterviewBit**

Given a matrix of integers **A** of size **N x M** and an integer **B**. Write an efficient algorithm that searches for integer **B** in matrix **A**.
This matrix A has the following properties:

1. Integers in each row are **sorted** from left to right.
2. The first integer of each row is greater than or equal to the last integer of the previous row.

Return **1** if **B** is present in **A**, else return **0**.
**NOTE:** Rows are numbered from top to bottom, and columns are from left to right.

**Ques 7 -** **Monk and Nice Strings | Practice Problems**

Monk's best friend Micro's birthday is coming up. Micro likes Nice Strings very much, so Monk decided to gift him one. Monk is having *N* nice strings, so he'll choose one from those. But before he selects one, he need to know the Niceness value of all of those. Strings are arranged in an array *A*, and the Niceness value of string at position *i* is defined as the number of strings having position less than *i* which are lexicographicaly smaller than A[i]. Since nowadays, Monk is very busy with the Code Monk Series, he asked for your help.
**Note:** Array's index starts from *1*.

**Ques 8 -** **Monk and Sorting Algorithm | Practice Problems**

Monk recently taught Fredo about sorting. Now, he wants to check whether he understood the concept or not. So, he gave him the following algorithm and asked to implement it:
*Assumptions*:
We consider the rightmost digit of each number to be at index 1, second last at index 2 and so on till the leftmost digit of the number.
Meaning of ith chunk: This chunk consists of digits from position $5*i$ to $1+5*(i-1)$ in the given number.If there is no digit at some position in the number, take the digit to be 0.
Initially, *i* is *1*.

- Construct the ith chunk, for all of the integers in the array. Let's call the value of this chunk to be the weight of respective integer in the array.
- If weight of all the integers of the array is *0*, then STOP
- Sort the array according to the weights of integers. If two integers have same weight, then the one which appeared earlier should be positioned earlier after the sorting is done. The array changes to this sorted array.
- Increment *i* by *1* and repeat from STEP *1*

**Ques 9 -** **Monk being monitor | Practice Problems**

Monk being the monitor of the class needs to have all the information about the class students. He is very busy with many tasks related to the same, so he asked his friend Mishki for her help in one task. She will be given heights of all the students present in the class and she needs to choose *2* students having heights h1 and h2 respectively, such that h1>h2 and difference between the number of students having height h1 and number of students having height h2 is maximum.
**Note:** The difference should be greater than *0*.

**Ques 10 - Monk and Suffix Sort | Practice Problems**

Monk loves to play games. On his birthday, his friend gifted him a string *S*. Monk and his friend started playing a new game called as Suffix Game. In Suffix Game, Monk's friend will ask him lexicographically kth smallest suffix of the string *S*. Monk wants to eat the cake first so he asked you to play the game.